CYB 601: Software and Systems Security

Week – 1
Foundations:
Thinking like a security engineer

by

Aatiz Ghimire

Week 1: Thinking like a security engineer

- Today's Agenda:
  - Anderson's frame
  - Who attacks us
  - Eight design principles
  - How to map a system
- Lab 1: Build your VM in code, then map what you built.

Anderson's framework: Policy, mechanism, assurance, incentives

- Policy:
  - What you are trying to achieve.
  - This is the overarching rule or goal.
  - e.g., "Only authorized employees can access customer financial records"
- Mechanism:
  - The physical or technical controls built to enforce the policy.
  - e.g., firewalls, multi-factor authentication, cryptography, or locked server room doors.

Anderson's framework: Policy, mechanism, assurance, incentives

- Assurance:
  - The degree of confidence that your mechanisms actually work as intended under stress.
  - e.g., penetration testing, code audits, and vulnerability scanning.
- Incentives:
  - The economic and psychological motivations driving human behavior.
  - This explains why attackers attack, why defenders defend, and why regular users might bypass security controls for the sake of convenience.

Who is the opponent?

- Spies:
  - States with large budgets and patience.
  - e.g. SolarWinds cyber attack by Russian Foreign Intelligence Service (SVR), APT29, Cozybear
- Crooks:
  - Organized, profit-driven, industrialized, including dishonest insiders.
  - e.g. LockBit ransomware gangs

Who is the opponent?

- Geeks: Researchers and hobbyists who find and report flaws.
- The swamp: Hate campaigns, bullying, and intimate-partner abuse.

Saltzer and Schroeder's design principles (1975)

- Economy of mechanism
- Fail-safe defaults
- Complete mediation
- Open design
- Separation of privilege
- Least privilege
- Least common mechanism
- Psychological acceptability

Engineering trustworthy systems (NIST SP 800-160)

- Security is an emergent property: Like performance or reliability, it must be architected from day one, not bolted on before deployment.

- Trustworthiness is the goal: The system must behave predictably under stress, component failure, or active attack.

- Continuous Assurance: A single yearly penetration test is insufficient; security requires ongoing, verifiable validation.

Engineering trustworthy systems (NIST SP 800-160)

- Requirements, design, verification, and evidence all count: Every phase of the system life cycle carries equal weight.

- Evidence over claims: Infrastructure-as-code, automated scans, and SBOMs serve as your formal, mathematical proof of security.

Software, system or incentive failure?

- **Software failures (The Code)**: A literal bug, logic error, or vulnerability in the codebase.
  - Example: Log4Shell (a Java string-parsing feature exploited for remote code execution).
  - The Fix: Patching or rewriting the code.
- **System failures (The Architecture)**: Safe individual components that create a toxic combination when integrated.
  - Example: Capital One 2019 (A misconfigured Web Application Firewall combined with an over-permissive AWS IAM role).
- The Fix: Re-architecting boundaries and enforcing least privilege.

Software, system or incentive failure?

- **Incentive & Process failures (The Human Element)**: Mechanisms exist but are ignored, delayed, or bypassed due to competing priorities or friction.
  - Example: Equifax 2017 (An Apache Struts patch was available but left unapplied for months).
  - The Fix: Changing organizational workflows, automating toil, and aligning security with user goals.

- The Engineering Rule: You cannot patch a human incentive problem with a firewall.

What to inventory

- Infrastructure & Compute: Every virtual machine, container, and cluster node, including the precise OS and kernel versions.
- Data Stores & Classification: Every database, object storage bucket, and volume, explicitly mapped to the data classification (e.g., PII, financial, public) it holds.
- Identities & Secrets: Human user accounts, machine service accounts, API keys, and exactly what privileges they hold.

What to inventory

- Attack Vectors: External entry points (load balancers, ingress controllers), internal listening ports, and all third-party software dependencies.

- The Golden Rule of Cloud: Automation is the only source of truth. A spreadsheet maintained by hand is a liability, not an inventory.

Data-flow diagrams for security

- The Four Elements:
  - External Entities: Users, browsers, or external APIs.
  - Processes: Application code, web servers, or functions (e.g., Traefik, Juice Shop).
  - Data Stores: Databases, caches, or state files (e.g., Postgres, Terraform .tfstate).
  - Data Flows: Directional network traffic (HTTP, SQL queries, SSH).

Data-flow diagrams for security

- Trust Boundaries (The Red Lines): Drawn wherever there is a change in privilege level, data ownership, or network zone.
- The "Invisible" Attack Surface: Your CI/CD pipeline, hypervisor (Proxmox), jump hosts (bastion), and code repositories (GitHub) are all in scope.
- Foundation for Week 2: Today we map the architecture; next week we use this diagram to build a formal threat model.

The Kubernetes attack surface

- The Control Plane (API Server): The central brain of the cluster; anyone holding a valid kubeconfig has the keys to the kingdom.
- Nodes & Supply Chain: Kubelet agent access on worker nodes; the container images deployed and the external registries they are pulled from.
- Identity & Routing: Service accounts, RBAC (Role-Based Access Control) bindings, Ingress controllers, and internal Services.
- Network Defaults (The Flat Network): By default, there are no internal boundaries. Every pod can route traffic to every other pod without restriction.

SBOMs: know what you ship

- What it is: A Software Bill of Materials (SBOM) is a formal, machine-readable inventory listing every software component, library, and dependency inside an image.
- The Analogy: The "ingredients label" for your software.
- Tooling: syft generates SBOMs by inspecting compiled images.
- Standard Formats: CycloneDX (OWASP) or SPDX (Linux Foundation).
- The Value: Answers the question "Are we affected?" in minutes rather than weeks during a zero-day crisis.

CIS benchmarks and kube-bench

- The Translation Layer: The Center for Internet Security (CIS) Benchmark translates generic hardening advice into specific, testable configuration checks.
- The Testing Tool: *kube-bench* is an open-source tool that executes these checks directly against the cluster architecture.
- The Profile: Checks are highly specific to the distribution; we use the profile specifically calibrated for k3s.
- The Value of a Baseline: Today's score is merely a starting point. A baseline only matters if you actively remediate the failures and rerun the scan.

Infrastructure as code with Proxmox

- Declarative Infrastructure: Terraform reads your .tf configuration files and translates them into execution calls against the Proxmox API.
- The Core Workflow:
  - plan: Calculates and shows the exact infrastructure delta.
  - apply: Executes the required changes.
  - state: Records the deployed reality.
- The Git Golden Rules:
  - NEVER commit: State files (.tfstate) and API keys.
  - ALWAYS commit: The lock file (.terraform.lock.hcl).

Lab 1 walkthrough

- Provision, deploy, inventory, scan, baseline, diagram, rank assets, sign the rules.

- Deliverable: *lab1/* folder and a one-page risk memo, tagged *lab1-submit*.

This week's other work

- Discussion: Classify a recent breach and name the principle that would have changed it.
- Google Skills: A Tour of Google Cloud Hands-on Labs, with the boundary reflection.
- Rules of engagement: Signed and committed before any scan.

Thank you!
Any Questions?